SPRING MVC FRAMEWORK

- **A Spring MVC is a Java framework which is used to build web applications.**

- **It follows the Model-View-Controller design pattern. It implements all the basic features of a core spring framework like Inversion of Control, Dependency Injection.**

Advantages of Spring MVC Framework

- **Clear separation of parts – The MVC pattern divides the app into three parts:**
  - **Model (business logic/data)**
  - **View (UI)**
  - **Controller (input handling)**
    This makes the code cleaner and easier to manage.

- **Light-weight - It doesn't need a heavy server; runs on a simple servlet container like Tomcat.**

- **Rapid development - Different developers can work on Model, View, and Controller at the same time(parallel development), making development faster.**

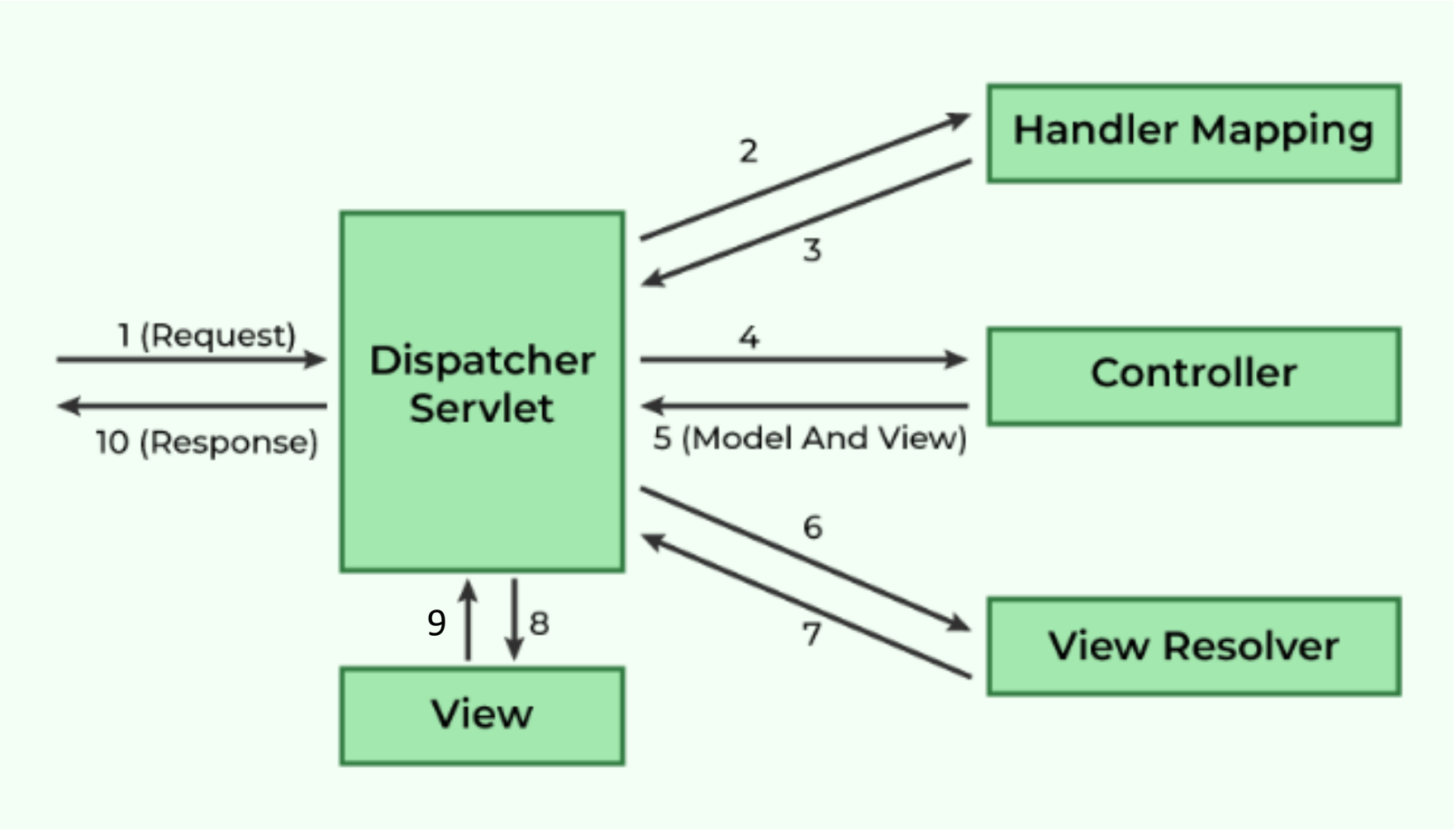
- **Reusable code - Spring manages and reuses using dependency injection, so you don't need to create them manually.**
- **Easy to test - In Spring, Dependency injection is used which makes application loosely coupled. Hence , testing is easy.**

- **Flexible Mapping - We can use simple annotations to connect URLs to methods easily.**

MVC : Model View Controller : is a design pattern that helps us organize code in a clean and structured way while building web applications.

- **Model**
  - The Model handles data and business logic.
  - It talks to the database, gets or updates data, and sends it back when needed.
    - Example: Entity classes (like User, Product), DAO classes (which connect to the database).
    - 👉 Think of it as "the brain" — it knows the data and how to handle it.
- **View**
  - The View shows the data to the user.
  - It's the UI part — like web pages or forms where users can enter or see information.
    - Example: JSP pages, HTML files.
    - 👉 Think of it as "the face" — it's what the user sees and interacts with.
- **Controller**
  - The Controller handles user requests and inputs.
  - It receives the request from the View, asks the Model for data if needed, and then decides which View to show.
    - 👉 Think of it as "the manager" — it controls how everything works together.

Spring MVC Architecture



- **Spring MVC Framework follows the Model-View-Controller architectural design pattern which works around the Front Controller i.e. the Dispatcher Servlet.**

Dispatcher Servlet

- **Dispatcher Servlet is the front controller that handles and dispatches all the incoming HTTP requests to the appropriate controller.**

- **When any web requests are made by the user, the requests will come first to the front Controller which is nothing but the Dispatcher Servlet.**

- **The dispatcher servlet accepts the requests and decides which controller method will be suitable to handle these requests by using the handler mapping.**

Handler Mapping :

- It Maps the request to a specific controller (e.g., using annotations like @RequestMapping . The Dispatcher Servlet then dispatches the HTTP request to the mapped controller.

Controller :

- The controller(mapped controller) will accept the request , process the request, interacts with the DAO layer(if necessary), and updates the Model. The controller then returns the Model data and a logical view name to the DispatcherServlet.

View Resolver:

- The DispatcherServlet uses View Resolver.
- The ViewResolver resolves the logical view name into an actual view (like /WEB-INF/views/success.jsp).
- The resolved view is given to the DispacterServlet.
- Then the DispatcherServlet sends back the view(JSP file) to the user as a response.

Creating a simple spring MVC project

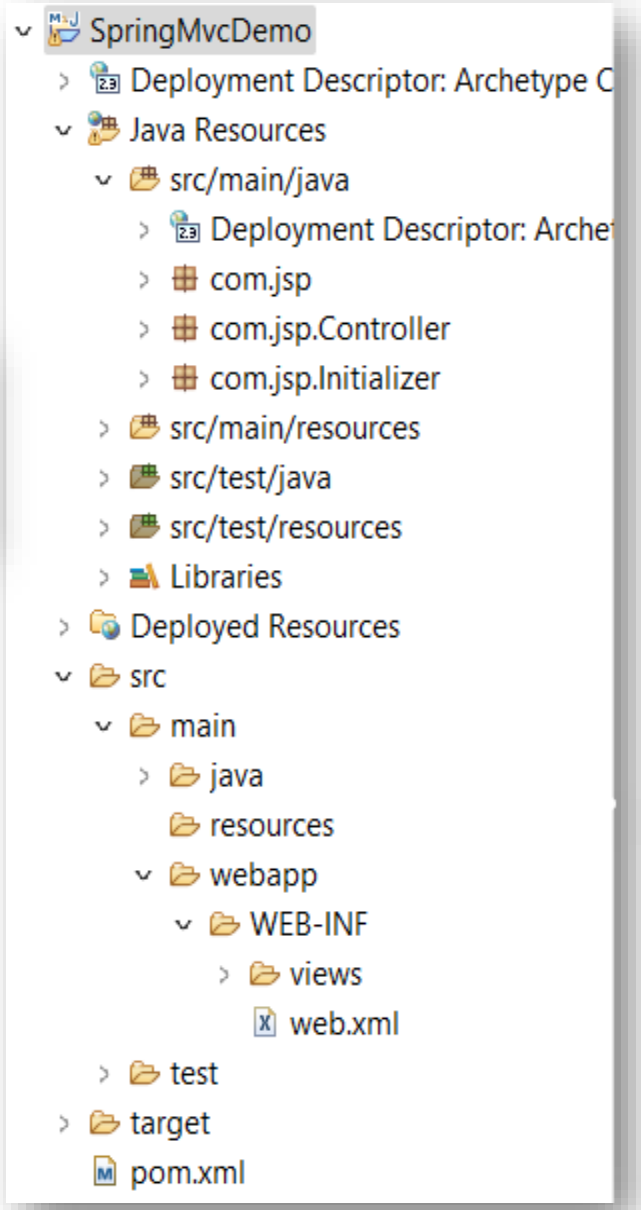
1. **Create a maven web project by Specifying group ID and artifact ID.**
2. **Once project is created , in pom.xml file , add spring web mvc , servlet and JSP dependency.**

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-webmvc</artifactId>
    <version>5.3.18</version>
</dependency>
```

```
<dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>javax.servlet-api</artifactId>
    <version>3.1.0</version>
    <scope>provided</scope>
</dependency>
```

```
<dependency>
    <groupId>javax.servlet.jsp</groupId>
    <artifactId>jsp-api</artifactId>
    <version>2.1</version>
    <scope>provided</scope>
</dependency>
```

1. **Configure the maven project to get all missing folders(build path configuration).**
2. **Create packages in the following way : com.jsp (base package to configure bean creation) , com.jsp.initializer (sub package to create DispatcherServlet) and com.jsp.controller (sub package for creating the controller class.**
3. **For creating views(web pages): in webapp folder , we have WEB-INF folder. Inside WEB-INF folder, create a sub folder "views"(folder name can be anything). The project structure is given here for your reference.**



NOTE : webapp folder is directly accessible by the browser. Hence , we will not create web pages inside webapp folder directly. Instead , we will configure the ViewResolver to look inside WEB-INF/views folder as The contents of WEB-INF(any file inside WEB-INF) are protected from direct access through the browser.

Step 6: In com.jsp package , create the configuration java class annotated with @Configuration and @ComponentScan. This configuration class indicates that specified package and all its sub packages must be scanned for components (object creation).

We configure the view resolver in configuration class by creating a method which creates and returns an object of InternalResourceViewResolver. This method should be annotated with @Bean to specify that the method produces a bean to be managed by the Spring container.

InternalResourceViewResolver

- InternalResourceViewResolver is a built-in Spring MVC view resolver.
- It is used to map logical view names(file names like "success", "error", "delete") returned by controllers to actual JSP files located in the project.
- We configure the InternalResourceViewResolver To automatically append a prefix and suffix to the view name and forward the request to the correct JSP file. This Makes it easier for developers to avoid writing full JSP paths everywhere.

```java
@Configuration
@ComponentScan(basePackages = "com.jsp")
public class DemoConfigure {

    @Bean
    public InternalResourceViewResolver getViewResolver()
    {
        InternalResourceViewResolver viewResolver = new InternalResourceViewResolver();
        viewResolver.setPrefix("/WEB-INF/views/");
        viewResolver.setSuffix(".jsp");
        return viewResolver;
    }
}
```

Here, @Bean annotation specifies that the method is returning objects which should be specified (registered) as a spring Bean in IOC container. So these objects will be now managed by IOC container and whenever we want these objects , we can ask IOC container to inject them.

- **Prefix: /WEB-INF/views/**
  - **This means all your JSPs are stored inside this folder, which is not directly accessible from the browser (a security best practice).**
- **Suffix: .jsp**
  - **When a controller returns "delete", Spring adds .jsp as a suffix to the returned file name and resolves it as:**

/WEB-INF/views/delete.jsp

Step 7 : In com.jsp.initializer package , create DispatcherServlet class by extending AbstractAnnotationConfigDispatcherServletInitializer class and override the methods in following manner :

```
public class DemoInitializer extends AbstractAnnotationConfigDispatcherServletInitializer {

    @Override
    protected Class<?>[] getRootConfigClasses() {
        // TODO Auto-generated method stub
        return null;
    }

    @Override
    protected Class<?>[] getServletConfigClasses() {
        // TODO Auto-generated method stub
        return new Class[] { Config.class };
    }

    @Override
    protected String[] getServletMappings() {
        // TODO Auto-generated method stub
        return new String[] { "/" };
    }

}
```

This method is to specify the configuration class for DIspatcherServlet. The configuration class provides information about packages that needs to be scanned for beans.

This method tells Spring which URL patterns should be handled by the DispatcherServlet.
{"/"}; --> **this means : Dispatcher Servlet Handles all requests(common mapping).**

{"/hi"}; --> **this means : Dispatcher Servlet will handle requests sent to /hi URL only. In such case , Dispatcher Servlet will not handle requests sent to any other URL pattern such as - "/login" , "/reg" etc.**

Step 8: **In com.jsp.controller package , create a controller class annotated with @Controller annotation.**

In controller class , we will be creating request handler methods. These handler methods are methods annotated with annotations like @RequestMapping etc and have specific URLs.

These methods handle incoming HTTP requests, process them, and return a response.

In controller class , here we create a method which returns view name (filename) as a response.

```
@Controller
public class DemoController {

    @RequestMapping("/hi")
    public String get()
    {
        return "index";
    }
}
}
```

URL to send request to this request handler method.

Step 9 : Whenever user will give the URL as /hi , request will be sent from DispatcherServlet to this method. This method will accept the request , and as a response , we are returning index file to the user.

So create index.jsp file in views folder and write whatever data you want to display on the browser.

- **When a controller returns a view name, the DispatcherServlet uses the configured ViewResolver to convert that name into the actual view file. Since we have configured the view Resolver , It will look in the /WEB-INF/views/ folder (as per the prefix) and adds .jsp (as per the suffix) to find the file and if file is present , it returns it to the user as a view.**

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
<h1>Hello , this is my first MVC project</h1>
</body>
</html>
```

Step 10: Now run the project , right click on project name → run as → run on server → select tomcat server → finish.

Once the server starts , it will launch the browser and you will get 404 error.

Specify the URL in the URL bar : http://localhost:8080/springMvcDemo/hi

NOTE : In controller class , multiple request handler methods (i.e.. Multiple mappings) can be created where each handler method can handle different types of requests..

Whenever we create a handler method , these methods must always return view (the page that needs to be displayed on the browser).

To bind data using model (to send data from controller to JSP file) :

- the handler methods in controller will always return view.
- From controller , along with returning the JSP page as view , we can bind data using model and send it to JSP page.

- All the steps from 1 to 6 are same.

Step 7 : In controller class, we will create a handler method and bind data using model.

The method then returns view name i.e..JSP file name as response.(The model also gets transferred to the JSP page, where we can access the model data).

```
@Controller
public class DemoController {

    @RequestMapping("/hello")
    public String sayHello(Model model) {
        model.addAttribute("myname", "Sheela");
        return "welcome";
    }

}
```

Declare a Model type variable for Binding(storing) data. Holds data in the form of key-value pairs.
This data is accessible in the view.

addAttribute(String attributeName , Object attribute value)

Attribute name and value/object is added to the model using addAttribute() method.
Attribute name will be used in JSP page for accessinig the model data.

step 9 : Whenever user will give the URL as /hello , DispatcherServlet will send the request to sayHello handler method which will accept the request.

It will the add the specified data/object into the model and return welcome file name as response . View resolver will then resolve the file name into actual file and return welcome.jsp page.

So we will now create welcome.jsp page.

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8" isELIgnored="false" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>

<h1>WELCOME TO MY PAGE</h1>
<h1>MY NAME IS ${myname}</h1>

</body>
</html>
```

To evaluate the expression

Here , we will write whatever data we want to display to the user.
To access the model data sent from the controller , we use ${ attributeName} .
${attributeName} is an expression. If we want this expression to be evaluated , then we need to declare isELIgnored = "false" .
Otherwise , it will be treated as plain text and will be ignored.

To return model and view together :

ModelAndView is a class provided by Spring MVC to encapsulate the data (Model) and view (View name) that a controller method returns to the framework. It combines these two components into a single object.

- **When a Spring MVC controller returns a ModelAndView object , the framework uses it to:**
  - **Render a specific view.**
  - **Provide the view with data stored in the model.**

All the steps from 1 to 7 are same.

Step 8 : In controller class, we will create a handler method which returns ModelAndView Object. In this method , we can bind data/object with the model and set the view and return both together.

```
@Controller
public class DemoController {

    @RequestMapping("/display")
    public ModelAndView display() {

        ModelAndView mv = new ModelAndView();
        mv.addObject("myname", "Ananya");
        mv.setViewName("show");
        return mv;
    }
}
```

step 9 : Whenever user will give the URL as /display , DispatcherServlet will send the request to display handler method which will accept the request.

It will the add the specified data/object into the model and set the view name, for example display. It then returns both model and view as a response.

So we will now create display.jsp page.

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8" isELIgnored="false"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>

<h1>WELCOME ${myname}</h1>

</body>
</html>
```

Accessing the model data sent from the controller and displaying the data to the user.